

Project 2: Frequent Subtrees Mining

Yassine Zeamari - Noma : 67522000 - group : 53

April 2026

1 Introduction

In this project, we focus on mining frequent subtrees from a database of trees.

A subtree keeps the parent-child relationships from the original tree. Compared to sequences, trees are more complex because they have a hierarchical structure.

The goal is to find all subtrees that appear frequently in the dataset, according to a minimum support threshold.

We also study the performance of the algorithm in terms of execution time, memory usage, and number of patterns found.

2 Adapting Sequence Mining to Trees

We adapted a pattern-growth approach, inspired by PrefixSpan, to handle tree-structured data.

In sequence mining, patterns are extended by adding items to a sequence. In our case, patterns are trees, so the extension is done by adding a new node to the current tree.

To ensure a consistent exploration, we only extend patterns along their rightmost path. This allows us to systematically generate all possible tree structures without redundancy.

Unlike sequences, trees require preserving parent-child relationships. Therefore, we represent each pattern using two arrays: one for node labels and one for parent indices.

Support computation is also different. Instead of checking if a sequence appears in another sequence, we must verify whether a tree pattern is an induced subtree of another tree.

3 Implementation

Each tree is represented using two lists: a list of labels and a list of parent indices. This representation allows efficient navigation of the tree structure.

To explore the search space, we use a depth-first search (DFS) strategy. Starting from frequent patterns of size 1, we recursively extend patterns by adding new nodes.

To avoid generating duplicate patterns, we store already visited patterns in a set using their structural representation.

Support computation is performed by scanning the dataset and checking subtree inclusion using the `is_valid_subtree` function.

4 Optimizations

Several optimizations were implemented to improve performance.

First, we avoid exploring patterns that are not frequent, which reduces the search space significantly.

Second, we store visited patterns to prevent redundant computations.

Finally, we use a recursive matching function for subtree verification, which ensures that unnecessary comparisons are avoided during support computation.

5 Experimental Evaluation

5.1 Setup

We evaluated the algorithm on the two provided datasets: `toy` and `small`.

We measured the following metrics:

- execution time,
- number of patterns,
- memory usage.

We tested several values of the minimum support threshold.

5.2 Results

5.2.1 toy Dataset

Table 1: Frequent pattern mining results

minsup	patterns	time (s)	memory (KB)
0.90	1	0.0303	23.97
0.80	8	0.0273	37.95
0.70	8	0.0389	34.91
0.60	11	0.1005	54.02
0.50	11	0.2133	51.22
0.40	15	0.2488	71.57
0.30	15	0.1776	70.30
0.20	25	0.2656	169.48
0.10	25	0.2743	137.96

5.2.2 small Dataset

Table 2: Frequent pattern mining results

minsup	patterns	time (s)	memory (KB)
0.90	1	0.0022	25.31
0.80	1	0.0016	24.52
0.70	1	0.0046	24.49
0.60	1	0.0041	24.53
0.50	1	0.0028	24.50
0.40	1	0.0017	24.48
0.30	2	0.0022	24.45
0.20	3	0.0077	24.42
0.10	5	0.0062	24.39

6 Analysis of Results

For the `toy` dataset, when the minimum support decreases, the number of patterns increases significantly. For example, the number of patterns grows from 1 ($\text{minsup} = 0.90$) to 25 ($\text{minsup} = 0.10$). This increase also leads to a higher execution time, since more candidate patterns need to be evaluated.

Memory usage follows the same trend. When more patterns are generated, more memory is required to store them and to manage the recursive exploration.

This shows that the algorithm scales poorly when the minimum support decreases, due to the rapid increase in the number of candidate patterns.

For the `small` dataset, the behavior is different. The number of patterns remains very low, even for small values of `minsup`. As a result, both execution time and memory usage remain almost constant.

This shows that the performance of the algorithm strongly depends on the number of frequent patterns.

7 Conclusion

We implemented an algorithm to mine frequent induced subtrees.

The results show that performance strongly depends on the minimum support threshold. Lower thresholds produce more patterns but increase runtime and memory usage.

The main cost comes from support computation, which requires checking each pattern in all trees.

This behavior is typical in pattern mining problems, where the search space grows quickly when constraints are relaxed.